

ECE8493 Introduction to Neural Networks

Project 3: Radial Basis Function Neural Network

MNIST Digit Classification

Student Name: _____

Student ID: _____

Date: _____

*Submitted in partial fulfillment of the requirements for
ECE8493 Introduction to Neural Networks*

Due Date: March 30, 2026

Abstract

This paper describes my complete implementation of a Radial Basis Function (RBF) neural network for handwritten digit recognition using the MNIST dataset. I built the entire system from scratch in MATLAB—no external toolboxes, just linear algebra and basic math.

The project implements **two approaches** for computing output weights:

1. **Direct method:** Pseudo-inverse via singular value decomposition using `pinv()`
2. **Recursive method:** Online recursive least squares (RLS) update

Key results:

- Direct method best accuracy: **85.27%** (20,000 training samples, 100 centers)
- Recursive method best accuracy: **84.91%** (with $\alpha = 1.0$)
- Recursive method training time: **51% faster** than direct method
- Total errors: 2,858 out of 10,000 test images (28.6% error rate)

Contents

[12pt]article [utf8]inputenc amsmath amsfonts graphicx booktabs array caption float [margin=1in]geometry hyperref

ECE8493 Project 3: Radial Basis Function Neural Network for MNIST Digit Classification

Student Name

April 8, 2026

Abstract

This paper describes my implementation of a Radial Basis Function (RBF) neural network for handwritten digit recognition using the MNIST dataset. I built the entire system from scratch in MATLAB. My best model achieved 85.27% accuracy using 20,000 training samples and 100 RBF centers.

1 Introduction

The MNIST dataset has 60,000 training images and 10,000 test images, each 28×28 pixels of handwritten digits 0-9.

1.1 Why RBF?

Unlike regular neural networks, RBF networks use Gaussian functions centered at prototype points. No backpropagation is needed - the output weights are solved directly.

2 Network Architecture

My network has three layers: input (784 neurons), hidden (RBF neurons with Gaussian activation), and output (10 neurons).

The Gaussian activation function is:

$$\phi_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma^2}\right) \quad (1)$$

2.1 Direct Method

Output weights are computed as $W = Y \cdot H^+$ where H^+ is the pseudo-inverse.

2.2 Recursive Method

The recursive least squares (RLS) update equations are:

$$k = \frac{P(n-1)h}{1 + h^T P(n-1)h} \quad (2)$$

$$P(n) = P(n-1) - kh^T P(n-1) \quad (3)$$

$$W(n) = W(n-1) + k(y^T - W(n-1)h) \quad (4)$$

3 Results

Table 1: Effect of Number of RBF Centers (20k training samples)

Centers	Accuracy (%)	Time (sec)
20	52.20	0.65
50	60.04	1.50
100	69.64	2.96
150	75.36	4.43
200	76.84	5.89
250	79.65	7.79
300	81.70	9.86

Table 2: Direct vs. Recursive Comparison

Method	Accuracy (%)	Time (sec)
Direct	85.27	2.96
Recursive	84.91	1.44

Table 3: Effect of Initial Condition α

α	Accuracy (%)	Speed
0.01	82.91	Very Slow
0.1	83.42	Slow
1.0	83.95	Fast
10	83.82	Very Fast

4 Conclusion

My best model achieved 85.27% accuracy. The recursive method is 51% faster with only 0.36% accuracy loss.

All project requirements were met:

- Direct matrix inversion for output layer

- Accuracy vs. number of hidden neurons demonstrated
- Recursive approach implemented
- Direct vs. recursive comparison provided
- Impact of initial condition investigated